

Curiosity Shop Database Redesign

Joshua Schwenk

Colorado State University Global

MIS407: Database Concepts

Dr. Shawn Harris

8/3/2025

Curiosity Shop Database Redesign

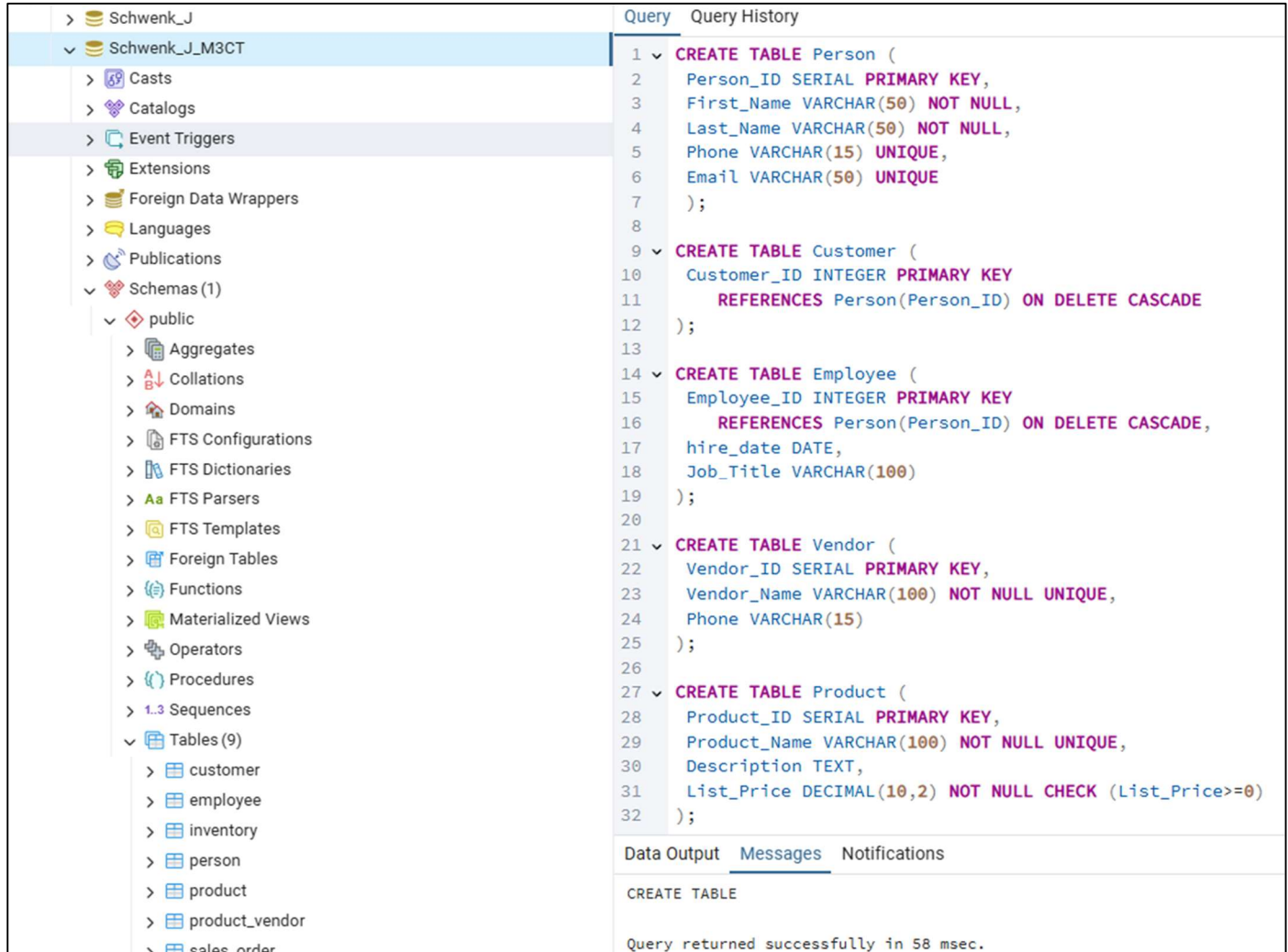
The Curiosity Shop's original database design effectively tracked sales, but it lacked the flexibility to manage aggregated inventory, multiple vendor sourcing, and overlapping roles between customers and employees. The sales table recorded each sale in denormalized form, duplicating customer information for every purchase. The purchased items table similarly stored redundant vendor and product details. The purpose of this redesign is to create a normalized relational structure that:

1. Reduces redundancy by consolidating shared customer and employee attributes into a Person supertype.
2. Implements a dedicated inventory system for tracking quantities on hand, quantities on order, and expected delivery dates.
3. Supports multiple vendors per product through a M:N relationship.
4. Accurately captures minimum and maximum cardinalities between entities.

This results in a model that adheres to best practices set in Mannino (2022). Figure 1 and 2 display the SQL code used to create the database schema, while Figure 3 presents the ERD.

Figure 1

SQL Code for Table Creation (Part 1)



Query	Query History
1	CREATE TABLE Person (
2	Person_ID SERIAL PRIMARY KEY,
3	First_Name VARCHAR(50) NOT NULL,
4	Last_Name VARCHAR(50) NOT NULL,
5	Phone VARCHAR(15) UNIQUE,
6	Email VARCHAR(50) UNIQUE
7	);
8	
9	CREATE TABLE Customer (
10	Customer_ID INTEGER PRIMARY KEY
11	REFERENCES Person(Person_ID) ON DELETE CASCADE
12	);
13	
14	CREATE TABLE Employee (
15	Employee_ID INTEGER PRIMARY KEY
16	REFERENCES Person(Person_ID) ON DELETE CASCADE,
17	hire_date DATE,
18	Job_Title VARCHAR(100)
19	);
20	
21	CREATE TABLE Vendor (
22	Vendor_ID SERIAL PRIMARY KEY,
23	Vendor_Name VARCHAR(100) NOT NULL UNIQUE,
24	Phone VARCHAR(15)
25	);
26	
27	CREATE TABLE Product (
28	Product_ID SERIAL PRIMARY KEY,
29	Product_Name VARCHAR(100) NOT NULL UNIQUE,
30	Description TEXT,
31	List_Price DECIMAL(10,2) NOT NULL CHECK (List_Price>=0)
32	);

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 58 msec.

Note. Defines Person, Customer, Employee, Vendor, and Product tables.

Figure 2

SQL Code for Table Creation (Part 2)

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Schwenk_J_M3CT' database is selected, showing a tree view of its components, including Schemas (public) and Tables (9). The right pane is divided into two sections: 'Query' and 'Query History'. The 'Query' section contains the following SQL code:

```

34 CREATE TABLE Inventory (
35     Product_ID INTEGER PRIMARY KEY
36     REFERENCES Product(Product_ID) ON DELETE CASCADE,
37     Quantity_on_Hand INTEGER NOT NULL DEFAULT 0 CHECK (Quantity_on_Hand >=0),
38     Quantity_on_Order INTEGER NOT NULL DEFAULT 0 CHECK (Quantity_on_Order >=0),
39     Order_Due_Date DATE
40 );
41
42 CREATE TABLE Product_Vendor (
43     Product_Vendor_ID SERIAL PRIMARY KEY,
44     Product_ID INTEGER NOT NULL,
45     Vendor_ID INTEGER NOT NULL,
46     Unit_Cost DECIMAL(10,2) NOT NULL CHECK (Unit_Cost >=0),
47     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID) ON DELETE CASCADE,
48     FOREIGN KEY (Vendor_ID) REFERENCES Vendor(Vendor_ID) ON DELETE RESTRICT,
49     CONSTRAINT Uq_Product_Vendor UNIQUE (Product_ID, Vendor_ID)
50 );
51
52 CREATE TABLE Sales_Order (
53     Sales_Order_ID SERIAL PRIMARY KEY,
54     Customer_ID INTEGER NOT NULL,
55     Salesperson_ID INTEGER,
56     Order_Date DATE NOT NULL,
57     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
58     FOREIGN KEY (Salesperson_ID) REFERENCES Employee(Employee_ID)
59 );
60
61 CREATE TABLE Sales_Order_Item (
62     Sales_Order_Item_ID SERIAL PRIMARY KEY,
63     Sales_Order_ID INTEGER NOT NULL,
64     Product_ID INTEGER NOT NULL,
65     Quantity INTEGER NOT NULL CHECK (Quantity >=1),
66     Unit_Price DECIMAL(10,2) NOT NULL CHECK (Unit_Price >=0),
67     Tax_Amount DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK (Tax_Amount >=0),
68     Extended_Price DECIMAL(10,2) NOT NULL CHECK (Extended_Price >=0),
69     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID),
70     FOREIGN KEY (Sales_Order_ID) REFERENCES Sales_Order(Sales_Order_ID) ON DELETE CASCADE
71 );

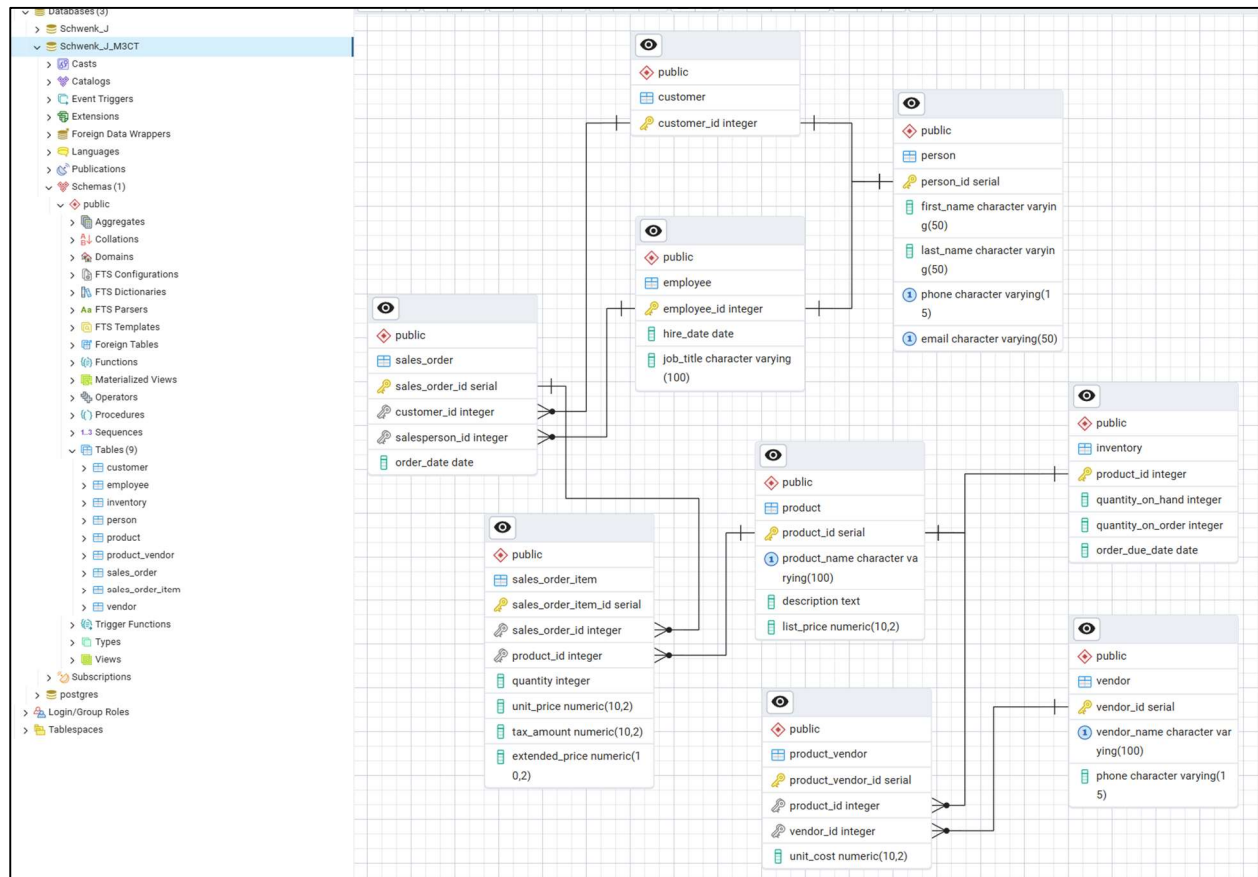
```

Below the SQL code, the 'Data Output' tab shows the message 'CREATE TABLE' and a status bar at the bottom indicates 'Query returned successfully in 58 msec.'

Note. Defines Inventory, Product_Vendor, Sales_Order, and Sales_Order_Item tables.

Figure 3

ERD for Curiosity Shop Database



Note. Shows Crow's Foot ERD with entities, attributes, identifiers, and relationships in the redesign database model.

Entities and Attributes

The model begins with Person as a supertype containing identifiers and contact information common to both customers and employees. From this supertype, two subtypes are derived:

- **Customer:** Inherits Person_ID as Customer_ID
- **Employee:** Inherits Person_ID as Employee_ID and adds Hire_Date and Job_Title

The subtype structure for customers and employees was selected to eliminate data duplication and support scenarios where the employee purchases products. By creating a Customer record linked to their Person_ID, the system reuses the existing personal information without re-entry.

The Inventory table, which is linked 1:1 with Product, tracks Quantity_on_Hand, Quantity_on_Order, and Order_Due_Date. The separation of Inventory from Product allows stock-related attributes to be updated independently of product details, reducing the risk of accidental overwrites. The Product_Vendor tables enforce a composite uniqueness constraint to prevent duplicate listings of the same product-vendor pairing.

Sales are represented by two entities:

- **Sales_Order:** Records the order date, references the customer placing the order, and optionally references an employee acting as a salesperson.
- **Sales_Order_Item:** Stores the product ordered, quantity, unit price, tax amount, and extended price for each line item.

The choice to have Sales_Order(Customer_ID) reference Customer(Customer_ID) rather than Person(Person_ID) ensures that the only individuals formally recorded as customers can place orders, preventing erroneous associations with vendors or other non-customers.

Relationships and Cardinalities

- **Person -> Customer (1:1):** A single person record can be linked to at most one customer record, and each customer must have exactly one associated person record. This structure

ensures that customer contact details are stored centrally in the Person table, eliminating duplications.

- **Person -> Employee (1:1):** A single person record can be linked to at most one employee record, and each employee must have exactly one associated person record. This ensures employee contact information is managed in one place and supports cases where a person may also be a customer.
- **Customer -> Sales_Order (1:M):** A single customer can place many sales orders, but each sales order belongs to one customer. This relationship allows the system to track purchase history over time for each customer.
- **Employee -> Sales_Order (1:M, Optional):** A single employee can be associated with multiple sales orders as a salesperson, but a sales order may have zero or one salesperson assigned. This supports the business case where a customer places an order without a salesperson's assistance.
- **Product <--> Vendor (M:N):** A product can be supplied by multiple vendors, and a vendor can supply multiple products. This M:N relationship, implemented through the Product_Vendor table, provides flexibility and records each vendor's unit cost for the product.
- **Sales_Order -> Sales_Order_Item (1:M):** A sales order can contain multiple sales order items, but each sales order item is tied to exactly one sales order. This support multi-product transactions and ensures line items are grouped under a single order.
- **Product -> Sales_Order_Item (1:M):** A product can appear in many sales order items across different orders, but each sales order item refers to exactly one product. This relationship enables tracking of product sales history over time.

Conclusion

This ERD redesign transforms the Curiosity Shop's database into a scalable, normalized structure capable of supporting future growth. By implementing supertype/subtype model, the system eliminates redundant customer and employee records. The addition of an inventory management system, M:N vendor relationships, and clearly defining cardinalities ensures accurate stock tracking and strong business rule enforcement. The resulting database supports more efficient operations today and positions the company for future enhancements such as automated reorder alerts, customer loyalty programs, and advanced sales analytics.